

What Actually Fills a Context Window

The API is stateless, tool results are almost all of it, and the effective window is smaller than the advertised one.

The Messages API is **stateless**. You resend the entire conversation on every single turn.

What is in every request

COMPONENT	BEHAVIOUR OVER A SESSION
System prompt	Fixed. Paid for on every turn.
Tool definitions	Fixed. Paid for on every turn, whether used or not.
Memory files and loaded context	Fixed. Loaded once per session, resent every turn.
Conversation history	Grows. Every message you have exchanged.
Tool results	Grows fastest. Every result the agent has ever received.

ON THE EXAM

The fixed parts set your floor. The growing parts decide how long the session can run.

In one of Anthropic's own research agents, file-read results were **96%** of the context. Reasoning was under 2%.

Where the tokens actually go

- ◆ **Tool results dominate.** A file read, a query result, a search response. Each one lands whole.
- ◆ **Verbose tools are the worst offenders**, especially anything that lists rather than searches.
- ◆ **Tool definitions are a standing charge.** Forty definitions cost you on every turn of every conversation.
- ◆ **The messages themselves are rarely the problem.** Optimise what enters, not what is said.

ON THE EXAM

Module 5's address book was about this. A tool that dumps rather than filters is a context problem wearing a tool costume.

The **effective** window is smaller than the advertised one. Attention degrades well before the limit.

How degradation shows up

- ◆ **It forgets a constraint from earlier** that is still, technically, in the window.
- ◆ **Its later decisions get worse** while its earlier ones were fine.
- ◆ **It repeats work** it has already done, or contradicts a conclusion it reached.
- ◆ **Nothing errors.** There is no exception and no warning, just a system quietly getting less good.

ON THE EXAM

"Its answers get vaguer as the session goes on" is the exam's phrasing for this, almost every time.

Clearing, or compaction?

TOOL RESULT CLEARING

Drop it wholesale

- Removes stale tool calls and their results
- The content is gone until re-fetched
- Cheap: no inference cost
- The lightest-touch option

COMPACTION

Summarise and restart

- Runs the whole transcript through the model
- Substance survives; verbatim detail does not
- Costs an inference call
- Fires at a threshold you configure

THE TIEBREAKER

Clearing loses the content but keeps it re-fetchable. Compaction keeps the substance but loses the exact wording.

What survives compaction, and what does not

- ◆ **Survives: high-level facts central to the task.** Key figures, main entities, major comparisons.
- ◆ **Lost: obscure specifics.** A cell in an appendix table, a statistic nobody flagged, exact phrasing.
- ◆ **Aggressive compaction loses context whose importance only becomes clear later.**
- ◆ **Tune the prompt for recall first, then precision.** Capture everything, then trim the superfluous.

ON THE EXAM

Compression ratios of a hundred to one are achievable on tool output. What you keep is a prompt-design decision.

Move it out of the window

- ◆ A scratchpad file holds findings, decisions and progress outside the context window entirely.
- ◆ The agent writes as it goes and reads back only what it needs.
- ◆ It survives compaction, because it was never in the transcript to be compacted.
- ◆ It survives the session, which is the difference between clearing and memory.

ON THE EXAM

Clearing and compaction manage what is inside the window. External memory moves things out of it.

Keep it in context, or write it down?

IN CONTEXT

Immediately available

- No extra call to retrieve it
- Full detail, exact wording preserved
- Costs tokens on every subsequent turn
- Vulnerable to compaction and clearing

ON DISK

Retrieved when needed

- Costs nothing until it is read
- Survives compaction and the session
- Needs a deliberate read to use
- Only as good as what was written down

THE TIEBREAKER

Keep what the next decision needs. Write down what a later decision might need.

Extract, do not accumulate

- ◆ **A verbose tool result arrives whole** and stays whole, for the rest of the session.
- ◆ **Pull the facts out of it and keep those.** Three fields instead of three thousand tokens.
- ◆ **Better: fix the tool**, so it returns the three fields in the first place.
- ◆ **Best: let code consume the intermediate output** and return only the processed result to the model.

ON THE EXAM

The cheapest token is the one that never enters the window. Module 5's response design is context management.

Subagents, from the context angle

- ◆ **A subagent has its own window.** Whatever it reads stays inside it.
- ◆ **Only its final message returns**, so a huge exploration becomes a short summary.
- ◆ **Anthropic's research system returns roughly one to two thousand tokens per subagent.**
- ◆ **It costs a round trip and a context load.** Not free, and often much cheaper than the alternative.

ON THE EXAM

Module 3 called this isolation. From Domain 5's side it is a context strategy, and it is often the strongest one.

Currency check: this is now first-party

TOPIC	EXAM ANSWER	CURRENT REALITY
Managing a filling window	Summarise manually, use scratchpad files	Context editing clears stale tool results automatically.
Cross-session state	Write it to a file yourself	A memory tool primitive exists on the platform.
Auto-compaction	Not covered	Claude Code auto-compacts near the top of the window.

ON THE EXAM

The exam-era techniques still work and are still the right answer. They are now also available as platform primitives.

Reading the stem: context questions

WHAT THE STEM SAYS	WHAT IT WANTS
"answers get vaguer as the session goes on"	Degradation. Reduce what is in the window.
"context fills after a few tool calls"	Tool response design, not a bigger model
"it forgot a decision from earlier"	Summarise, or write decisions to a file
"we moved to a larger context window and it still happens"	Structural. A bigger window fills at the same rate.
"exploration generates output we do not need"	A subagent, or a forked skill
"it needs this next week, not next turn"	External memory, not context

ON THE EXAM

A bigger window buys time. It never changes the rate at which the window fills.

Module 15 in five lines

- ◆ The API is stateless. Everything is resent every turn, so nothing is free.
- ◆ Tool results are almost all of it. Optimise what enters, not what is said.
- ◆ The effective window is smaller than the advertised one, and degradation is silent.
- ◆ Clearing loses content but keeps it re-fetchable. Compaction keeps substance and loses detail.
- ◆ Keep what the next decision needs; write down what a later decision might need.

ON THE EXAM

Five scenarios next. Two of them offer a bigger context window as the tempting wrong answer.

A research agent reads documents from a corpus to answer analyst questions. Traces show its context reaching 300,000 tokens within five turns.

Breaking the context down, file-read results account for the overwhelming majority. The conversation itself is a few thousand tokens.

The team is planning to move to a model with a larger context window.

What should you advise?

- ☐ A The larger window is the right move, since the corpus is genuinely large
- ☐ B Address the file-read results: read less, extract facts from what is read, or delegate reading to subagents
- ☐ C Shorten the system prompt and reduce the number of tool definitions
- ☐ D Summarise the conversation history every few turns

◆ PAUSE AND DECIDE

What should you advise?

- A

 Bigger window
Buys headroom at the same fill rate. Five turns becomes eight, and the structure is unchanged.
- C

 Trim the fixed parts
Real savings on a small share. Optimising 2% while ignoring the 96%.
- B

 Attack the file reads
The stem tells you where the tokens are. Fixing the dominant component is the only change that scales.
- D

 Summarise the conversation
The conversation is a few thousand tokens. Compressing it changes almost nothing.

THE SIGNAL

The stem gives you the breakdown. Options C and D both optimise the small parts, which is why the breakdown is there.

An agent handles long technical support sessions. Early in a session it correctly applies a customer-specific constraint the user mentioned: the customer is on a legacy plan and cannot use the new API.

Around forty turns in, it recommends the new API. The constraint is still present in the conversation history; nothing was truncated.

No error was raised at any point.

What is happening?

- ☐ A The constraint was silently dropped when the context window overflowed
- ☐ B Attention degrades as the window fills, so information present early receives less weight later
- ☐ C The model does not support constraints stated in user messages
- ☐ D The conversation history is not being resent, so early turns are missing

◆ PAUSE AND DECIDE

What is happening?

- A

 Overflow dropped it
Overflow raises an error. The stem says nothing was truncated and nothing errored.
- B

 Attention degradation
Present but under-weighted is exactly this failure. It is silent, gradual, and worsens with length.
- C

 Constraints in user messages
Invented limitation. It applied the constraint correctly for the first thirty-something turns.
- D

 History not resent
Then it would have lost everything from early turns, not one constraint.

THE SIGNAL

"Still present in the history, nothing truncated, no error" is the full signature of degradation rather than overflow.

A long-running analysis agent hits its context limit during multi-hour sessions. The team implements compaction: at a token threshold, the transcript is summarised and the session reinitiated with the summary.

It works. But analysts report that specific figures cited earlier — a particular sample size, a footnoted caveat — go missing after a compaction, while the overall conclusions survive intact.

What is the most appropriate response?

- ☐ A Raise the compaction threshold so it fires less often
- ☐ B Write key figures and caveats to a file as they are found, so they survive outside the transcript
- ☐ C Disable compaction and accept hitting the context limit
- ☐ D Instruct the compaction prompt to preserve everything

◆ PAUSE AND DECIDE

What is the most appropriate response?

- A

 Higher threshold
Fewer compactions, same loss each time. It delays the problem on long sessions.
- B

 Write them outside the transcript
What is not in the transcript cannot be compacted away. It targets exactly what is being lost.
- C

 Disable compaction
Trades a lossy session for one that stops entirely. Worse on multi-hour work.
- D

 Preserve everything
A summary that preserves everything is the transcript. Compaction that does not compress is not compaction.

THE SIGNAL

"Conclusions survive, specifics are lost" is exactly how compaction is documented to behave. It is working correctly.

A codebase agent answers questions about a large repository. To answer anything it calls a tool that returns the full contents of every file in a directory, then reasons over them.

After three or four questions its context is nearly full and answers become noticeably worse. The team has already moved to a larger model once, which delayed the problem by about two questions.

Most questions concern one or two files.

What is the most effective change?

- ☐ A Move to an even larger context window
- ☐ B Replace the directory-dump tool with search and targeted read, so only relevant files enter context
- ☐ C Summarise the file contents after each question
- ☐ D Start a fresh session after every question

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Even larger window
They already tried this and bought two questions. The stem records the experiment for you.
- B

 Search, then read what matters
Most questions need one or two files. Stopping the dump at source is the only change that scales.
- C

 Summarise after each question
The full contents already entered context. Summarising adds tokens on top of what is already there.
- D

 Fresh session each time
Solves context by discarding continuity. Every question restarts from nothing.

THE SIGNAL

The stem records a failed experiment: a bigger window bought two questions. That is there to eliminate option A for you.

An agent performs a multi-step migration audit. Step four needs a decision made in step one, and a list of affected modules discovered in step two.

The team wants this to work reliably even on very long audits where compaction will fire, and across audits that span more than one session.

Currently everything lives in the conversation.

What should they implement?

- ☐ A Increase the compaction threshold so the early steps stay in the transcript longer
- ☐ B Write decisions and findings to a state file as each step completes, and read from it in later steps
- ☐ C Repeat the step-one decision in every subsequent prompt
- ☐ D Run each step as a subagent so its context stays isolated

◆ PAUSE AND DECIDE

What should they implement?

- A

 Higher threshold
Delays compaction within one session. It does nothing across sessions, which the stem also requires.
- B

 A state file
Survives compaction and survives the session. Both requirements, one mechanism.
- C

 Repeat it every prompt
Works for one known decision. It does not scale to findings you cannot predict in advance.
- D

 Subagents per step
Isolates context and makes the problem worse. Subagents inherit nothing, so step four sees even less.

THE SIGNAL

Two requirements: survive compaction, and survive the session. Only external state satisfies both.